

Auto-NBA Parameter Experimentation

1st Jaden Ziga

2nd Jacob Wheeler

3rd Ryan Hastie

Abstract—This is a report covering five experiments done on the parameters within the two configuration files of the Auto-NBA AI search tool. These experiments were done in order to better understand what parameters a user of the Auto-NBA tool has control over, and how they can change these parameters to more effectively utilize Auto-NBA. The results of these experiments give a better understanding of how different parameters in the configuration files affect the search time of Auto-NBA, the training time of the found model, and how accurate the model is.

I. INTRODUCTION

Auto-NBA is a tool that searches for the optimal AI model across the joint space of network architecture, bitwidths, and accelerators/hardware [1]. The network architecture part of the search space involves searching for the most optimal arrangement of layers within the AI model. The bitwidth part of the search space refers to finding the best precision to use for numbers found within the model such as weights and biases. A higher bitwidth would mean more precise numbers leading to a more accurate model that consumes a higher amount of resources, whereas a lower bitwidth would have less precise numbers, a lower accuracy, and less resource requirements. The hardware/accelerator portion of the search space is searching the best accelerator design to pair with the found model and bitwidth. Auto-NBA chooses the optimal model in regards to both accuracy and efficiency. Accuracy refers to the performance of the found model and how correct it is in identifying the subject of images. Efficiency refers to how fast the model is able to run as well as how resource efficient it is. These two factors often clash where a higher accuracy means a lower efficiency, and a higher efficiency means a lower accuracy. Auto-NBA provides a way for these two factors to be balanced within the found model. The preference for efficiency versus accuracy can also be affected by the user via the configuration files. The Auto-NBA repository also includes files for training the model that was found to be most optimal during the search. The model being found and trained is a CNN which the configuration files allow to be searched and trained with the CIFAR-10, CIFAR-100, or ImageNet dataset in mind. The choice of the dataset is chosen in both the search configuration file `config search.py` and the training configuration file `config train.py` with the same dataset being chosen in each configuration file. These configuration files also contain many variables/parameters in

them that can be changed to affect the performance of Auto-NBA. Many of these parameters lack a clear explanation of the role they play in Auto-NBA and how changing these values affects the performance of the tool. As previously mentioned, the use of the Auto-NBA tool has two phases, the searching phase and the training phase. The search phase is where Auto-NBA is applied as the tool is how the optimal model is found. While the training phase is not unique to the Auto-NBA tool, it is critical in understanding how the found model is performing both in accuracy and runtime. The search phase sees Auto-NBA find the optimal AI model design from the previously mentioned joint search space of network architecture, bitwidths, and accelerators. It is in this search phase that the parameters changed in this paper's experiments are found.

II. METHODOLOGY

This section will outline the practices and methods that were used to change parameters within Auto-NBA and find the results of those changes. This paper details five experiments that were done on different Auto-NBA parameters. These parameters were chosen from the `config search.py` file as this is the key file affecting the performance of the Auto-NBA tool. The parameters from this file that were chosen for the experiments were parameters affecting the number of bitwidths in search space, the prioritization of accuracy vs efficiency, the number of filters within the layers, and the efficiency metric that is used. These parameters were chosen as they were likely to yield bigger differences in the accuracy and efficiency of the model. The effects of changing these parameters are measured in three ways being the amount of time that Auto-NBA searched for an optimal model, the amount of time it took to train the found model, and the reported accuracy of the trained model. Not all experiments are expected to show a change in all three measurements as some impact search time while others do not impact it in a meaningful way. Even if an experiment is not expected to affect a measurement, it was still collected to give a better overall understanding of the measurements and the parameters affect on the measurements. The process of running these experiments involved a Google Colab notebook that we had set up to run the Auto-NBA tool from the repository. Cells within this notebook involved pulling files from the Github repository, installing the proper libraries, solving issues with old dependencies, and running the actual files. The same notebook and cells were used for

every experiment as well as all experiments using the T4 GPU runtime. The configuration files were kept the same except for the parameter that was being changed between the different runs of each experiment. The search time measurement was collected at the end of the Auto-NBA search for the optimal model, and was determined via the printed timestamps for a more accurate measurement than the Google Colab cell itself which would often shorten runtime measurements to just the hour without the additional minutes. Similarly the training time was determined from the first and last timestamps that were printed during the training of the model. The accuracy measurement was collected from the printed accuracy for the epoch that training file found to have the best accuracy. These measurements were collected for each run of each experiment to allow for effective comparison between the runs of each experiment, as well as a general understanding of these values over the course of all experiments.

III. RESULTS

The first experiment that we ran targeted the bitwidth search space for Auto-NBA. In both the config search file and the config train file there is a parameter called C.num bits list. The number of bits list contains all of the bit widths that are searched for during the Auto-NBA search. For example if the line was C.num bits list = [4,6] then the Auto-NBA search space would search through models that use 4 or 6 as its bitwidth precision. The standard value, meaning the value that is in the Auto-NBA repo's configuration files, is [4,6,8]. For this experiment we used [4] for the first run, [4,6] for the second run, and [4,6,8] (the standard value) for the third run. These values were chosen for the first experiment in order to see how C.num bits list affects the search time as we believed that increasing the amount of values in the num bits list would increase the size of the search space and therefore increase the search time.

TABLE I
RESULTS OF EXPERIMENT 1

Configuration	Search Time	Train Time	Accuracy
[4]	55 min	46 min	77.917
[4, 6]	1 hr 35 min	40 min	81.647
[4, 6, 8]	2 hr 17 min	41 min	85.536

Figure 1: Shows the results of experiment 1

The results of the first experiment are shown within figure 1. As C.num bits list was configured to have more values we found that the search time increased from 55 minutes with one value, 1 hour 35 minutes for two values, and 2 hours 17 minutes for three values. This is in line with what we believed would happen, as there are more bitwidths added into Auto-NBA's search space, the search time would also increase. It showed an even more drastic change in search time than originally predicted as the search time more than doubled when increased from one value to three values. In addition to the search time increasing, the accuracy of the trained model also increased from 77.917 for one value, to 81.647 for two

values, and 85.537 for three values. The increase in accuracy of the trained model can be explained by the Auto-NBA search having more potential models to go through, increasing the chances of it finding a better model within that increased search space. For example with the list being [4] the bitwidth is always going to be 4 whereas for [4,6,8] it could also be 6 or 8 which would help to increase the accuracy due to the use of higher precision numbers. The train time showed no meaningful correlation with the change in configuration like we predicted might happen. Our original belief was that the found model would likely have a higher bitwidth precision in subsequent runs which would result in more precise numbers that would take up more memory and slow down the training time as well. This was not found to be the case and the results of experiment two is also in line with this observation. We are unsure exactly why there is no correlation, it is possible that the increase in bitwidth precision does not meaningfully impact the efficiency of the model, but it is also possible that the bitwidth precision is simulated and not actually used during this training.

Experiment 2 builds off of experiment one and also targets the C.num bits list that was targeted in the first experiment. The goal of this experiment was to target the specific bitwidths that we wanted without increasing the search time that was demonstrated in experiment 1. The configurations for this experiment were [4] for run 1, [6] for run 2, and [8] for run 3. The idea is that this would not increase the search space as each run only has 1 bitwidth, but that it would also show the accuracy increase seen in experiment one.

TABLE II
EXPERIMENT 2 RESULTS

Configuration	Search Time	Train Time	Accuracy
[4]	55 min	46 min	77.917
[6]	54 min	41 min	84.067
[8]	54 min	38 min	85.714

Figure 2: Shows the results of experiment 2

The results from figure 2 show that the search time was not affected by the differing configurations found in experiment 2. The search time was almost the same across all three runs which reflects the same size of the Auto-NBA search space across all experiment 2 runs. Using single values for the C.num bits list helps to cut the search time by about half, which when doing many Auto-NBA runs can end up saving great amounts of time. In addition to no change to search time, experiment 2 also showed an increase in accuracy with [4] resulting in 77.917, [6] resulting in 84.067, and [8] resulting in 85.714. This experiment confirms that each run increases in bitwidth precision as only the value specified is in the list. In experiment 1 it is possible that the found model for run three could have a bit width precision of 4, 6, or 8 as all values are present in the list. In this experiment however, we can be sure that the bitwidth increases in subsequent runs, matching the increase in accuracy. Using higher precision bitwidths gives more detailed weights, biases, etc., resulting in the increase

in accuracy amongst models Experiment 2 also reflects no strong correlation between configuration and train time akin to experiment 1. In this experiment however it also shows a decrease in training time between configurations. We believe this to just be a normal fluctuation in train time unconnected to the configuration changes. A 3 minute difference in training time as seen between runs 2 and 3 is not unheard of or unusual as we have noticed throughout our many Auto-NBA runs, so it is very possible that the decrease in subsequent training times is just a coincidence due to that normal fluctuation.

Experiment 3 targets a different parameter than experiments 1 and 2 as this experiment targets parameters within the config search file called alpha and beta. The default values for these parameters is having both alpha and beta set to 1. Run 1 keeps the default configuration, run 2 drops both values to 0.2, and run 3 drops both of them further to 0.05. We opted to move both alpha and beta in sequence as the default values are the same between alpha and beta. It is not clear exactly how alpha and beta figure into Auto-NBA but a comment in the configuration file describes them as “hardware cost weighted coefficients”. Because of this comment we believe that alpha and beta factor into the Auto-NBA math formulas that are used to determine how Auto-NBA searches for a model.

TABLE III
FIGURE 3: EXPERIMENT 3 RESULTS

Configuration	Search Time	Train Time	Accuracy
alpha 1, beta 1	2 hr 8 min	47 min	83.849
alpha 0.2, beta 0.2	2 hr 5 min	44 min	84.534
alpha 0.05, beta 0.05	2 hr 17 min	51 min	85.942

Figure 3: Shows the results of experiment 3

Experiment three showed no strong correlation between the alpha and beta configurations, and the search time. All three search times were within normal fluctuation times that we have seen in search time. All other parameters were kept to their default setting such as C.num bits list having [4,6,8] and the resulting 2 hour search time. The experiment parameters of alpha and beta for experiment 3 overall show no change in search time. Experiment three also showed no strong correlation between alpha and beta setting and the training time. It is possible that other configurations of alpha and beta such as values higher than 1 could affect the training time, but for our values they showed no trend in the training time. The accuracy did improve as alpha and beta were decreased, but the increase in accuracy was more subtle than the increase from the prior two experiments. It increased from 83.849 to 84.534 to 85.942. While the increases seen in experiment three are smaller than the other experiments, there is a clear trend for accuracy with no obvious downsides in search time and train time.

Experiment 4 targets the parameter C.efficiency metric that is found within the config search file. There is also a similar parameter within the config train file that we mirrored the change in, but the search parameter should be more crucial as the search phase is when Auto-NBA is prioritizing efficiency

vs accuracy, not the training phase. The efficiency metric can either be focused on the latency of the model or the flops, with flops being the amount of floating point operations that the model uses. We did two runs for experiment 4, one for latency and one for flops as those are the two values that this metric can use.

TABLE IV
FIGURE 4: EXPERIMENT 4 RESULTS

Configuration	Search Time	Train Time	Accuracy
latency	2 hr 15 min	38 min	84.266
flops	39 min	52 min	82.996

Figure 4: Shows the results of experiment 4

The difference in search time is stark, but it is not caused by how Auto-NBA uses the efficiency metric as it is instead from how Auto-NBA gets latency. In runs with latency as the metric, there are many iterations to determine bottlenecks which cause the search time to be much longer than a search time with flops as the metric. This is not from how the metric is being used in the actual search, it is simply due to these bottleneck iterations found within runs focused on latency. Train time also showed a stark difference with the latency run having a 38 minute train time and flops having a 52 minute train time. This is because during the latency run search, Auto-NBA is determining efficiency based on latency of the model rather than the flops which is more akin to how big the model is. Because of the latency prioritization, the found model has much better latency and thus has a much faster training time. The run for flops likely would take up much less space than the latency run, but that is not tracked by the three results we have kept track of in each experiment. There is also a small difference in the accuracy of the models, but it is not a big enough of a difference to be certain of the difference between latency and flops with regard to accuracy, without having more data from more runs.

Experiment 5 targets the parameter C.num channel list that is found within both the config search and config train files. This parameter targets the amount of filters within the convolutional layers of the found model, with the values in this list corresponding to different groups of convolutional layers within the found model. The standard value is [16,24,32,64,112,184,352], and we also ran it with values halved [8,12,16,32,56,92,176] and multiplied by 1.5 [24,36,48,96,168,276,528].

TABLE V
FIGURE 5: EXPERIMENT 5 RESULTS

Configuration	Search Time	Train Time	Accuracy
Standard [16, 24, 32, 64, 112, 184, 352]	2 hr 1 min	38 min	74.345
Halved [8, 12, 16, 32, 56, 92, 176]	1 hr 43 min	35 min	64.722
1.5x [24, 36, 48, 96, 168, 276, 528]	2 hr 21 min	35 min	78.611

Figure 5: Shows the results of experiment 5

The differing configurations showed an effect on search time with the halved values showing a lower search time than standard and the 1.5 values showing a higher search time. This

makes it clear that as the channel list values increase, so does the search time. This time increase is not shown in train time like it is in the search time as train time values were all within normal variation of training times. The accuracy of trained models also increased as the channel values increased with 64.722 for halved values, 74.345 for the standard, and 78.611 for the 1.5 values. This increase in accuracy can be attributed to the larger size of the model being able to capture more information about the cifar-10 images. This allows the model to better learn the images and thus be better at identifying unseen images. The lack of effect on train time is not in line with our expectations as we believed that the larger sized models would have a larger train time to pair with the better accuracy. Our expectations were shown to not be the case as the train time showed no strong correlation with the change in configurations of experiment 5. It is worth mentioning that there was also an experiment 6 that targeted the parameter C.num layer list but changing this parameter completely broke Auto-NBA. This failed experiment 6 highlights the goal of all these experiments which was to bring a better understanding to the Auto-NBA parameters in the configuration files in order to better configure the tool to its desired output.

IV. CONCLUSIONS

Overall the parameters we chose to experiment with showed an effect on search time, training time, and training accuracy. C.num bits list showed the ability to change the search time and accuracy, but not the training time. Having more values in this parameter increases the search space and thus search time, while having higher bitwidth precision results in a higher accuracy of the trained model. If the desired output of the Auto-NBA tool is a higher accuracy, it is beneficial to have higher bitwidths within this parameter, and if the search time is too long, then having only a single value in the list parameter as this keeps the search time around an hour. The alpha and beta parameters showed an effect on the accuracy of the model with lower alpha and beta values increasing the accuracy of the model. Therefore if the desired output of the Auto-NBA is for a more accurate model, then it would be beneficial to lower the alpha and beta values within the config search value. The efficiency metric parameter can be configured for latency or flops, with latency showing a shorter training time than flops. If training time needed to be shorter, a latency configuration would be more beneficial than flops whereas a smaller model would be favored with flops. Search time is much longer in latency runs as the bottleneck iterations are present, so if search time needed to be shorter than a flops run would be better suited. C.num channel list can be configured depending on search time and accuracy preferences. If a shorter search time is valued, then lower values within this parameter would be better suited than higher values. If accuracy is more important than search time, then larger values in this parameter would be better suited than lower values. Overall these five experiments bring more clarity to the effects on Auto-NBA performance from changing the two configuration files that are present in the repository. This allows for better configuration

of the Auto-NBA tool which allows for better customization of runs for Auto-NBA depending on the desired outcome of the run.

REFERENCES

- [1] Y. Fu, Y. Zhang, Y. Zhang, D. Cox, and Y. Lin, "Auto-NBA: Efficient and Effective Search Over the Joint Space of Networks, Bitwidths, and Accelerators," in *Proc. ICML*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.06575>